Appointment Booking Marketplace

System Architecture

Inspired by Fresha. A two-sided marketplace connecting consumers with local service businesses (salons, spas, barbers, wellness studios, etc.) for online appointment booking. Payments are handled in person at the venue for v1 — no online payment processing.

---

1. Product Scope

Two distinct user-facing products sharing one backend:

Side	Who	Core jobs-to-be-done
Consumer app	People booking appointments	Search nearby businesses, view services/prices, pick a time slot, book, get reminders, pay at the venue, leave reviews
Business dashboard	Salon/spa owners & staff	Manage calendar, staff schedules, services & pricing, clients (CRM), amount-due display, marketing/reminders
Admin/internal	Platform team	Approve businesses, manage categories/cities, moderate reviews, handle disputes

Out of scope for v1 (don't build yet): online payments, inventory/retail POS, payroll, multi-currency accounting — add later once the core booking loop works.

2. High-Level Architecture



For a first build, you don't need literal microservices — a modular monolith (one codebase, clearly separated modules/folders per domain: booking, search, reviews) is faster to ship and easier to reason about. Split into real services later only if a specific module needs independent scaling (search and notifications are usually first).

3. Recommended Tech Stack

Layer	Recommendation	Why
Frontend (consumer + dashboard)	Next.js (React) + TypeScript	SSR/SSG for SEO-critical marketplace pages (city/category landing pages, like Fresha's), fast client-side nav for the booking flow
Styling	Tailwind CSS	Fast to build consistent, responsive UI
Mobile	React Native (later) or PWA first	Reuse booking logic; ship PWA before native apps
Backend API	Node.js (NestJS) + TypeScript	Shared language with frontend, strong module/DI structure mapping to domain modules
Database	PostgreSQL	Relational integrity matters (bookings, no-double-booking constraints)
Caching / locks	Redis	Session cache, slot-locking during checkout, rate limiting
Search	Elasticsearch/OpenSearch (or Postgres full-text + PostGIS to start)	Geo + text search across businesses/services; start simple, graduate at scale
File/image storage	S3-compatible storage + CDN (CloudFront)	Business photos, staff photos
Payments	None for v1 — "Pay at store"	Customer pays the business directly in person; no PCI scope, no payout logic. Add online payments later as an opt-in per business.
Notifications	Twilio (SMS), SendGrid/Postmark (email), FCM (push)	Booking confirmations, reminders, cancellations
Background jobs	BullMQ (Redis-based queue)	Reminder scheduling, email digests
Infra	Docker + managed platform (Render/Railway/Fly.io for MVP, AWS/GCP later)	Ship fast now, scale later without a rewrite
Auth	NextAuth or Clerk/Auth0	Don't roll your own auth for an MVP
Observability	Sentry (errors) + Pino logging + Postgres query metrics	Catch booking-flow bugs early — this must never silently fail

4. Core Data Model (simplified)

```
User (id, name, email, phone, role: consumer|business_owner|staff|admin)
Business (id, owner_id, name, category, address, lat, lng, timezone, status)
Location (id, business_id, address, lat, lng)     -- for multi-location businesses
Staff (id, business_id, user_id, title, working_hours)
Service (id, business_id, name, duration_minutes, price, category)
StaffService (staff_id, service_id)                -- which staff perform which services
Availability (staff_id, day_of_week, start_time, end_time)  -- recurring hours
TimeOff (staff_id, start_datetime, end_datetime)   -- exceptions/vacation
Booking (id, consumer_id, business_id, staff_id, service_id,
         start_datetime, end_datetime, status: pending|confirmed|
         completed|cancelled|no_show, price)   -- price shown for reference only, paid in person
Review (id, booking_id, consumer_id, business_id, rating, comment)
Notification (id, user_id, type, channel, sent_at, status)
```

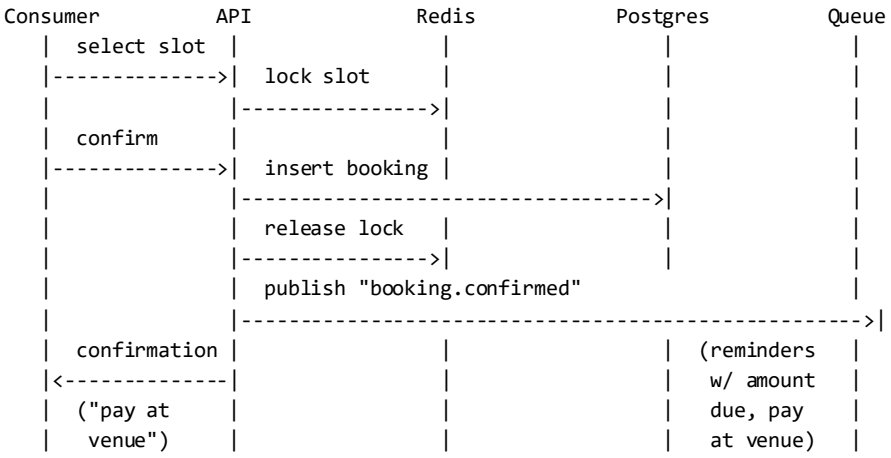
Key constraints that matter more than they first appear:

- No double-booking: a DB-level exclusion constraint (Postgres EXCLUDE USING gist on staff_id + time range) is far more reliable than only checking in application code.
- Timezones: store all datetimes in UTC; store each business's IANA timezone separately and convert at display time.

5. Critical Flow: Booking a Slot (avoiding double-booking)

This is the part of the system where correctness matters most — even without payments in the loop, two customers must never be able to grab the same slot.

1. Consumer selects service + staff + date → API returns available slots (computed from Availability minus TimeOff minus existing Bookings).
2. Consumer picks a slot and confirms → backend places a short-lived Redis lock on (staff_id, start_datetime) for ~30–60 seconds while the booking is created.
3. Booking row inserted inside a DB transaction, with the exclusion constraint as a final guarantee against race conditions.
4. Lock released; confirmation event published to the queue → triggers SMS/email confirmation ("Pay in person at the venue") and reminder jobs at 24h and 2h before, restating the amount due.
5. If the lock expires before the booking is written (e.g. client disconnects), the slot returns to the available pool automatically.



No-show protection without online payment: since there's no card on file to charge a no-show fee, lean instead on (a) requiring phone/email verification to book, (b) a "3 strikes" flag on a consumer's account after repeated no-shows, visible to businesses or auto-blocking further bookings, and (c) the 24h/2h reminder texts — the single biggest lever for reducing no-shows even without payment friction.

6. Search & Discovery (the Fresha homepage pattern)

Fresha's homepage is built around programmatic SEO landing pages by category x city (e.g. "Hair Salons in Sydney"), plus a live geo-search. Recreate this with:

- A generated-route pattern: /[category]/in/[city] rendered via Next.js SSG/ISR, pulling live business counts and top-rated listings for that combination.
- A geo-search endpoint: GET /businesses?category=&lat=&lng=&radius= using PostGIS ST_DWithin (simple) or Elasticsearch geo-queries (at scale).

- Cache popular city/category pages aggressively (ISR with revalidation, e.g. every 10–60 min) since business listings don't change second-to-second.

---

7. Business Dashboard Essentials

Minimum viable feature set to match Fresha's core value prop for businesses:

- Calendar view (day/week) per staff member, drag-to-reschedule
- Service & pricing management
- Client list with booking history (basic CRM)
- Amount-due display per booking (no online payment processing — settled in person)
- Automated SMS/email reminders (the single biggest reason businesses switch to platforms like this — no-show reduction)
- Basic reporting: bookings this week, revenue, no-show rate

---

8. Suggested Build Order (MVP → v1)

1. Auth + business onboarding (single location, single staff member to start)
2. Service & availability management for a business
3. Public booking flow for one business (no search yet) — prove the booking-without-double-booking flow end to end, with a clear "Pay at store" notice on confirmation
4. Notifications/reminders (huge value, moderate effort, and doing double duty as no-show mitigation since there's no deposit — don't leave this too late)
5. Search & discovery (multi-business, geo + category)
6. Reviews
7. Multi-staff, multi-location support
8. Admin panel for approving businesses/moderating content
9. Online payments (optional, opt-in per business) — the natural place to add this later without reworking the booking core, since Booking.price already exists for reference

Building it in this order means you have a working, bookable single-business site very early, and the "marketplace" layer (search across many businesses) is added on top of a proven booking core rather than being solved simultaneously with it.
